FT9xx Series 32-Bit Programming Guide

Table of Contents

1. □□ (Introduction) . . . 1
   1.1. □□□□ (Conventions) . . . 1
      1.1.1. □□□□□□□□□ LaTeX□ . . . 1
      1.1.2. □□□□□□Schematic□□□□ SVG . . . 2
2. □□□□□ (Register Maps) . . . 2
3. □□□□□□ (Peripheral Programming) . . . 3
   3.1. UART □□□ . . . 3
4. □□□□ (Modular Content) . . . 3
   4.1. CTRL Register (32-bit) . . . 3
      4.1.1. Bitfield Diagram . . . 3
      4.1.2. Field Description . . . 3
      4.1.3. C Example (Read-Modify-Write) . . . 4
5. □□□□□ . . . 5
6. □□□□□ 32-bit Register Example . . . 5
   6.1. GPIO_MODE Register (0x00) . . . 5
      6.1.1. Register Layout (32-bit) . . . 5
      6.1.2. Field Definitions . . . 5
      6.1.3. C Access Example . . . 6
7. □□□□ (Timing Diagrams) . . . 7
8. □□□□ architect diagrams . . . 7

1. □□ (Introduction)

□□□□□□□□□□□□□□□□□□□□ MCU □□□□□□□□□□□

> □□□□□□□□□□□□□ VDD □□ 3.3V □□□□□

1.1. □□□□ (Conventions)

1.1.1. □□□□□□□□□□ LaTeX□

□□□□□□□□ HTML/PDF/Markdown □□□□□□□□□□□□□□□□□□ LaTeX/TeXLive □□□□□□□□

- □□□□ `latexmath` □ LaTeX □□□□

```
[latexmath]
++++
f_{PWM} = \frac{f_{clk}}{(Prescaler + 1) \times (Period + 1)}
++++
```

- □□□ AsciiMath□□□□□□□□□□□□□□□

  □□□□□AsciiMath□□

```
:stem: asciimath

[asciimath]
++++
f_(PWM) = f_(clk) / ((Prescaler + 1) * (Period + 1))
++++
```

- □□□□□□□

```
stem:[f_(PWM) = f_(clk) / ((Prescaler + 1) * (Period + 1))]
```

- □□□□□□□□□□□□□□

  - □□□□□ + □□□□□□□□□□□□□□□
  - □□□□□□□□□□ SVG/PNG □□□□□□□□□□□□□□

1.1.2. □□□□□□Schematic□□□□ SVG

□□□□□□□□ HTML/PDF □□□□□□□□□□□□□□□□□□□□□□

- □□□□□/□□/□□□□□□□□ **SVG**□□□□□□□
- □□□□□□□□□ PNG/JPG □□□□□□□□□□□□□□□□□□□□□□□□
- □□□□□□□□`schematic_top.svg`, `schematic_power.svg`, `schematic_io.svg`
- □□□□□□□□

```
.□□□□□□□□ (Power Schematic)
image::schematic_power.svg[width=95%,align=center]
```

2. □□□□□ (Register Maps)

Register Name	Offset	Access	Description
GPIO_MODE	0x00	R/W	□□ GPIO □□□□
GPIO_ODATA	0x04	R/W	□□□□□□□□

Register Name	Offset	Access	Description
GPIO_IDATA	0x08	R	□□□□□□□

3. □□□□□□ (Peripheral Programming)

3.1. UART □□□

□□□□□ **C** □□ □□□□□□□□□□

```
#include "mcu_uart.h"

void uart_init(uint32_t baudrate) {
    UART->BAUD = SystemCoreClock / baudrate; // □□□□□
    UART->CTRL |= UART_ENABLE;               // □□□□
}
```

4. □□□□ (Modular Content)

4.1. CTRL Register (32-bit)

Base Address: `0x4002_0000`
Offset: `0x00`
Access: R/W
Reset: `0x0000_0000`

4.1.1. Bitfield Diagram

□ *1. Bits 31..16*

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RESERVED (31:24)								SPEED (23:16)							

□ *2. Bits 15..0*

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PULL (15:12)				MODE (11:8)				EN (7)	INTEN (6)	KEY (5:0)					

4.1.2. Field Description

Bits	Field	Reset	Description
31:24	RESERVED	0x00	Reserved. Keep at reset value.
23:16	SPEED	0x00	Output speed. `0=Low`, `1=Med`, `2=High`, `3=VeryHigh`

Bits	Field	Reset	Description
15:12	PULL	0x0	Pull config. `0=None`, `1=Up`, `2=Down`
11:8	MODE	0x0	Mode. `0=Input`, `1=Output`, `2=AF`, `3=Analog`
7	EN	0	Enable. `0=Disable`, `1=Enable`
6	INTEN	0	Interrupt enable. `0=Off`, `1=On`
5:0	KEY	0x00	Write key. Must write `0x2A` to update MODE/PULL/SPEED.

4.1.3. C Example (Read-Modify-Write)

```c
#include <stdint.h>

#define REG32(addr) (*(volatile uint32_t *)(addr))

#define CTRL_BASE   0x40020000UL
#define CTRL_OFF    0x00UL
#define CTRL        REG32(CTRL_BASE + CTRL_OFF)

#define CTRL_SPEED_Pos   16u
#define CTRL_SPEED_Msk   (0xFFu << CTRL_SPEED_Pos)

#define CTRL_PULL_Pos    12u
#define CTRL_PULL_Msk    (0xFu  << CTRL_PULL_Pos)

#define CTRL_MODE_Pos    8u
#define CTRL_MODE_Msk    (0xFu  << CTRL_MODE_Pos)

#define CTRL_EN_Pos      7u
#define CTRL_EN_Msk      (1u    << CTRL_EN_Pos)

#define CTRL_INTEN_Pos   6u
#define CTRL_INTEN_Msk   (1u    << CTRL_INTEN_Pos)

#define CTRL_KEY_Pos     0u
#define CTRL_KEY_Msk     (0x3Fu << CTRL_KEY_Pos)
#define CTRL_KEY_VALUE   0x2Au

static inline void ctrl_config(uint32_t mode, uint32_t pull, uint32_t speed, uint32_t
en)
{
    uint32_t reg = CTRL;

    // Preserve reserved bits; only modify documented fields.
    reg &= ~(CTRL_MODE_Msk | CTRL_PULL_Msk | CTRL_SPEED_Msk | CTRL_EN_Msk |
CTRL_KEY_Msk);
    reg |= ((mode  & 0xFu)  << CTRL_MODE_Pos);
    reg |= ((pull  & 0xFu)  << CTRL_PULL_Pos);
```

```
    reg |= ((speed & 0xFFu) << CTRL_SPEED_Pos);
    reg |= ((en ? 1u : 0u)  << CTRL_EN_Pos);
    reg |= (CTRL_KEY_VALUE  << CTRL_KEY_Pos);

    CTRL = reg;
}
```

5. □□□□□

□□ PWM □□□□□□□□□□□

```
f_{PWM} = \frac{f_{clk}}{(Prescaler + 1) \times (Period + 1)}
```

use ASCII math, □□□□:

```
f_(PWM) = f_(clk) / ((Prescaler + 1) * (Period + 1))
```

6. □□□□□ 32-bit Register Example

6.1. GPIO_MODE Register (0x00)

6.1.1. Register Layout (32-bit)

Bits	Field	Reset	Description
31:16	RESERVED	0x0000	Reserved, must keep default value.
15:8	SPEED	0x00	Output speed selection.
7:4	PULL	0x0	Pull-up / pull-down configuration.
3:0	MODE	0x0	Pin mode selection.

6.1.2. Field Definitions

- **MODE** (Bits [3:0])
  - `0x0` = Input
  - `0x1` = Output
  - `0x2` = Alternate Function
  - `0x3` = Analog
- **PULL** (Bits [7:4])
  - `0x0` = No pull
  - `0x1` = Pull-up

  - `0x2` = Pull-down
- **SPEED** (Bits [15:8])
  - `0x00` = Low
  - `0x01` = Medium
  - `0x02` = High
  - `0x03` = Very High

6.1.3. C Access Example

```c
#include <stdint.h>

#define GPIO_BASE      (0x40020000UL)
#define GPIO_MODE_OFF  (0x00UL)
#define GPIO_MODE      (*(volatile uint32_t *)(GPIO_BASE + GPIO_MODE_OFF))

// Bit positions / masks
#define GPIO_MODE_MODE_Pos   0
#define GPIO_MODE_MODE_Msk   (0xFUL << GPIO_MODE_MODE_Pos)

#define GPIO_MODE_PULL_Pos   4
#define GPIO_MODE_PULL_Msk   (0xFUL << GPIO_MODE_PULL_Pos)

#define GPIO_MODE_SPEED_Pos  8
#define GPIO_MODE_SPEED_Msk  (0xFFUL << GPIO_MODE_SPEED_Pos)

static inline void gpio_mode_set(uint32_t mode, uint32_t pull, uint32_t speed)
{
    uint32_t reg = GPIO_MODE;

    reg &= ~(GPIO_MODE_MODE_Msk | GPIO_MODE_PULL_Msk | GPIO_MODE_SPEED_Msk);
    reg |= ((mode  & 0xFUL)  << GPIO_MODE_MODE_Pos);
    reg |= ((pull  & 0xFUL)  << GPIO_MODE_PULL_Pos);
    reg |= ((speed & 0xFFUL) << GPIO_MODE_SPEED_Pos);

    GPIO_MODE = reg;
}

static inline uint32_t gpio_mode_get(void)
{
    return GPIO_MODE;
}
```

Notes

- This register is 32-bit aligned, so offsets increment by 4 bytes (0x00, 0x04, 0x08...).

- Always preserve **RESERVED** bits (read-modify-write) unless the spec says otherwise.

7. □□□□ (Timing Diagrams)

□□□ SPI □□□□□□□□□□□□

```
{ "signal": [
  { "name": "SCK",  "wave": "p......" },
  { "name": "MOSI", "wave": "x.3456..", "data": ["A2", "A6", "A5", "A4"] },
  { "name": "MISO", "wave": "x......",  "data": ["D7"] },
  { "name": "CSN",  "wave": "1.0...1." }
]}
```

8. □□□□ architect diagrams

This diagram is drawn by plantuml code.

```
@startuml
skinparam componentStyle rectangle

package "MCU Core" {
  [Cortex-M Processor] as CPU
}

package "SPI Peripheral" {
  [Register Interface] as REG
  [FIFO Buffer] as FIFO
  [State Machine] as FSM
  [Baud Rate Gen] as BRG
}

node "External Flash" as Flash

CPU <-> REG : APB Bus
REG <-> FIFO
FIFO <-> FSM
BRG -> FSM
FSM <-> Flash : SPI Bus
@enduml
```

It can be done by inserting the image :

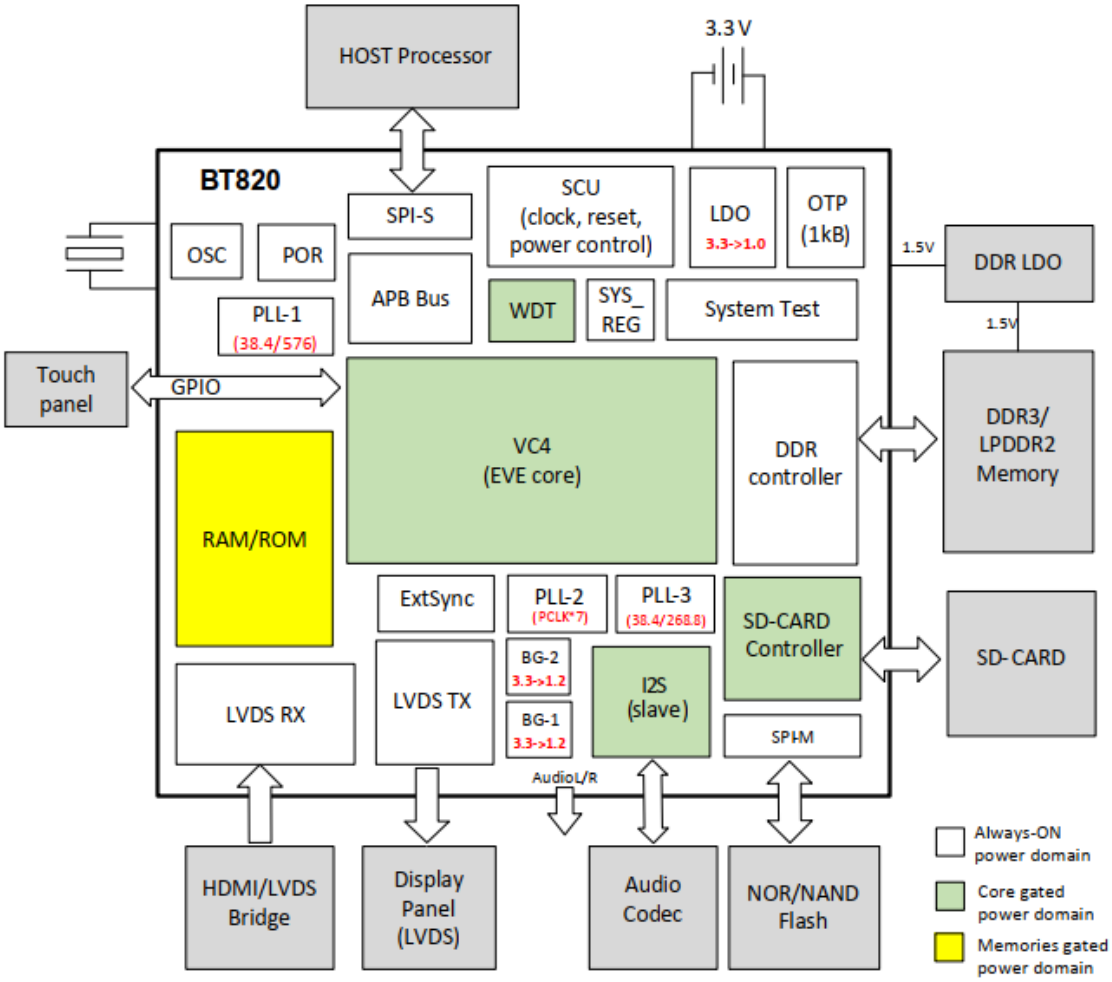

Figure □ 1. System Diagram (□□□□□□)

□□□□ (Revision History)

□□	□□	□□	□□□□
2026-02-12	v0.1	Paul Jiao	□□□□□□□□□□□□□□□□
2026-02-15	v0.2	KH Low	□□□□□□□ TOC □□